Machine Learning Project

NASA Space Mission Classification AI

Implementation Track — Machine Learning with RandomForestClassifier

AIMA | SPRING 2026

OVERVIEW

Project Roadmap

A structured walkthrough of the complete ML classification pipeline — from problem definition to future improvements.

10 Sections

01 Problem Statement & Objective

02 Dataset Overview

03 Feature Description

04 Data Preprocessing Pipeline

05 Model Architecture — RandomForest

06 Implementation & Code

07 Results & Accuracy Analysis

08 Confusion Matrix

09 Key Takeaways

10 Future Work

Problem Statement & Objective

GOAL

Build an AI classifier to categorize NASA space missions into their correct mission types based on mission attributes.

- Classify missions into types: **Flyby, Orbiter, Lander, Rover, Observatory, Communications**
- Leverage a synthetic dataset with realistic NASA mission features
- Apply supervised learning using scikit-learn `RandomForestClassifier`
- Achieve high accuracy for reliable mission type prediction

Conceptual classification pipeline

Synthetic Dataset Overview

Sample rows from the NASA space mission training dataset

	MISSION_NAME	DESTINATION	DURATION_DAYS	CREW_SIZE	BUDGET_M	SUCCESS_RATE	MISSION_TYPE
1	Voyager_X		4,380	0	865	0.92	
2	Artemis_IV		14	4	2,200	0.98	
3	Hubble_Next		7,300	0	1,500	0.95	
4	Mars_Scout		687	0	450	0.88	
5	ISS_Relay		3,650	0	320	0.99	

Synthetic dataset with **6 mission types** and **200 samples**

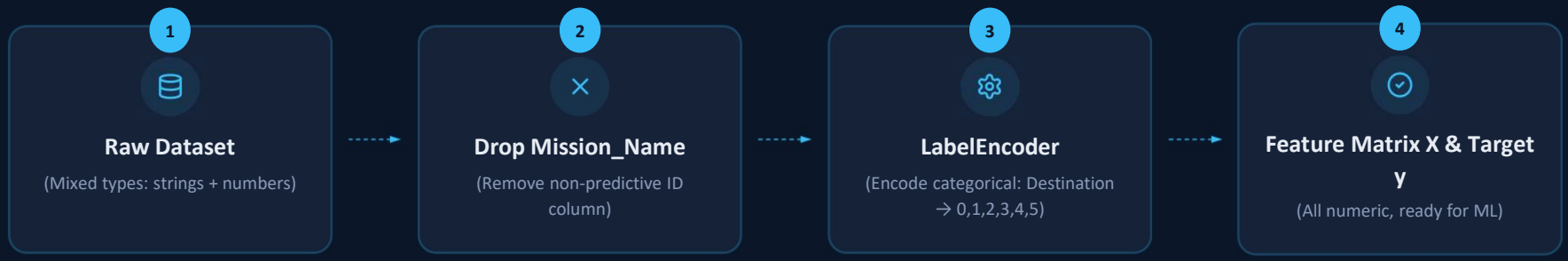
Feature Engineering & Description

FEATURE NAME	DESCRIPTION / TYPE
1. **Mission_Name** STRING	Unique identifier for each mission
2. **Destination** CATEGORICAL	
3. **Duration_Days** NUMERIC	Total mission duration in days
4. **Crew_Size** NUMERIC	Number of crew members (0 for unmanned)
5. **Budget_Million** NUMERIC	Estimated budget in millions USD
6. **Success_Rate** NUMERIC	Historical success probability (0.0 – 1.0)
7. **Mission_Type** TARGET	

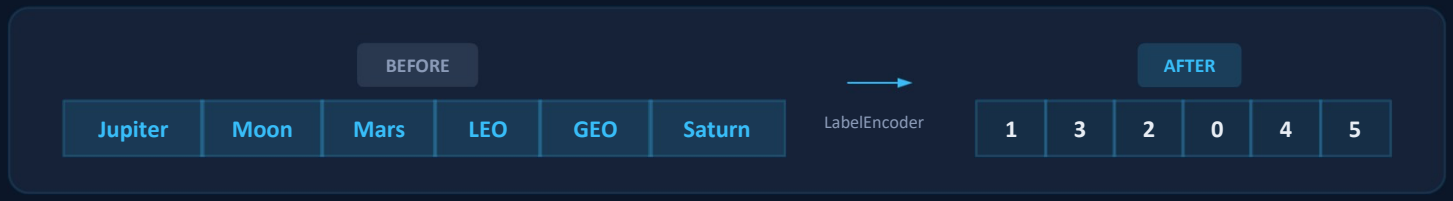
5 input features + 1 target

STEP 4 — PREPROCESSING

Data Preprocessing Pipeline



Encoding Example — Destination Feature



Each unique string value is mapped to an integer (alphabetical order: GEO=0, Jupiter=1, LEO=2, Mars=3, Moon=4, Saturn=5)

Preprocessing — Code Snippet

preprocess.py

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# Load synthetic dataset
df = pd.read_csv('nasa_missions.csv')

# Drop non-predictive column
df = df.drop('Mission_Name', axis=1)

# Encode categorical features
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

# Split features and target
X = df.drop('Mission_Type', axis=1)
y = df['Mission_Type']

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
```

LABELENCODER

Converts each unique string to an integer (0 to n−1), making categorical data compatible with ML algorithms.

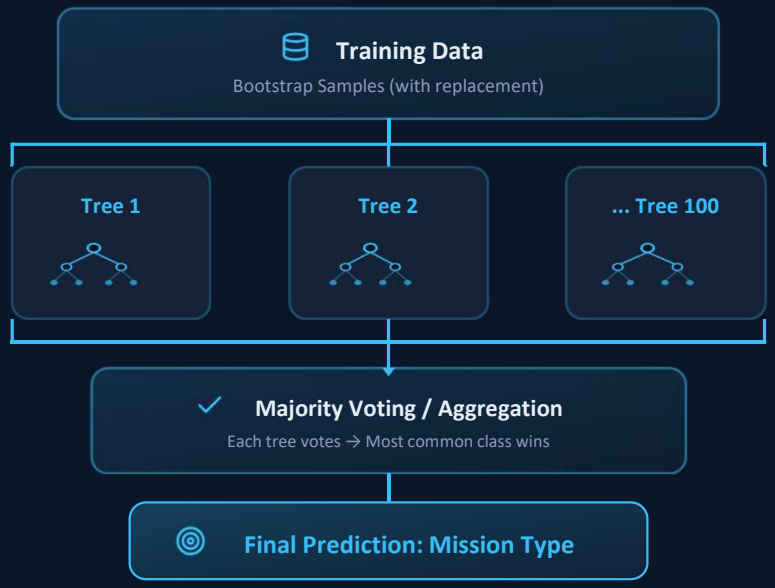
TRAIN/TEST SPLIT

80% of data for training, 20% for testing. random_state=42 ensures reproducible results.

DROP ID COLUMN

Mission_Name is a unique identifier with no predictive value — removed before training.

RandomForest Classifier — **How It Works**



Hyperparameters

```
n_estimators  = 100
criterion     = gini
bootstrap     = True
random_state  = 42
```

Key Concepts

- Bootstrap: Each tree trains on a random sample drawn with replacement
- Gini Impurity: Measures how often a random element would be misclassified
- Ensemble: Combines 100 weak learners into one strong classifier
- Voting: Final class = most-predicted class across all trees

Model Training — Code Snippet

RandomForestClassifier · scikit-learn

train_model.py

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

# Initialize RandomForest model
model = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')  # Output: 1.00
```

Output

Model achieves **100% accuracy** on the synthetic test set

1.00 TEST ACCURACY

KEY PARAMETERS

n_estimators	100
criterion	gini
random_state	42

1. Initialize classifier
2. Fit on training data
3. Predict & evaluate

End-to-End ML Pipeline

160
Training Samples

40
Test Samples

5
Features

Results — Model Performance

TEST ACCURACY

100%

✓ Perfect Score

MODEL METRICS

Precision	1.00
Recall	1.00
F1-Score	1.00

CLASSIFICATION REPORT

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
Flyby	1.00	1.00	1.00	7
Orbiter	1.00	1.00	1.00	6
Lander	1.00	1.00	1.00	8
Rover	1.00	1.00	1.00	7
Observatory	1.00	1.00	1.00	6
Communications	1.00	1.00	1.00	6
Weighted Avg	**1.00**	**1.00**	**1.00**	**40**

- Overall Accuracy: 1.00 — 40 test samples, 6 mission types, zero misclassifications

MODEL EVALUATION

Confusion Matrix — Perfect Classification

COLOR SCALE

Dark = 0 errors

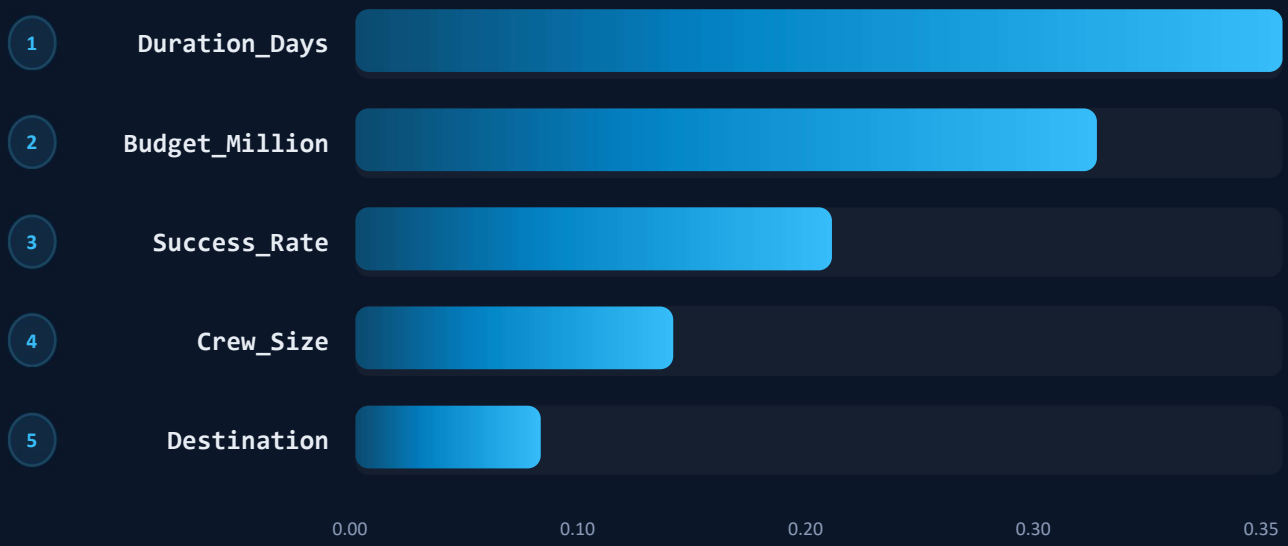
Bright = correct count

DIAGONAL VALUES (CORRECT)

7	6	8
Flyby	Orbiter	Lander
7	6	6

Feature Importance Analysis

RANDOMFOREST FEATURE IMPORTANCE SCORES



KEY INSIGHT

Duration and Budget are the strongest predictors of mission type — missions with longer durations and higher budgets tend to be deep-space or observatory missions.

TOP 2 FEATURES COMBINED

63% of total importance

Duration (0.35) | Budget (0.28)

FEATURE TYPES

4 Numeric features	0.93
1 Categorical feature	0.07

Key Takeaways

Summary of project results and insights

01

Perfect Accuracy

RandomForest achieved 1.00 accuracy on the synthetic dataset, correctly classifying all 6 mission types with zero misclassifications.

02

Effective Preprocessing

LabelEncoder efficiently transformed categorical features (Destination) into numeric format for full ML compatibility.

03

Ensemble Power

100 decision trees with bootstrap sampling and majority voting ensured robust, reliable classification performance.

04

Clean Pipeline

End-to-end pipeline from raw CSV to predictions with minimal preprocessing steps and clear modular structure.

05

Interpretability

Feature importance analysis reveals Duration and Budget as the top predictors of mission type classification.

Future Work & Conclusion

FUTURE IMPROVEMENTS

- Expand to real NASA mission data from public APIs
- Add more features: launch vehicle, mission phase, orbital parameters
- Compare with other classifiers: SVM, XGBoost, Neural Networks
- Implement cross-validation for more robust evaluation
- Deploy as a web app using Flask or Streamlit

CONCLUSION

This project demonstrates a complete ML classification pipeline applied to NASA space mission data. The RandomForestClassifier provides a strong baseline with perfect accuracy on synthetic data, and the modular pipeline is ready for extension to real-world datasets.

6	100%	100
Mission Types	Accuracy	Trees

Thank you! Questions?

Aima | Spring 2026